

Report 2: Data Mining Project

Group 10

Valentina Kloster , Livan Miranda Labrada, Mora Dominguez

Course: CAP 5771 Principles of Data Mining - Fall 2025

Institution: Florida International University

Date: November 2025

Introduction

The goal of this project is to build a reproducible machine learning pipeline that can classify foods into a binary target class based on product and nutritional information. Because the dataset contains multiple feature types (numeric nutritional values and categorical attributes), the workflow must include proper preprocessing, a controlled data split strategy, and fair comparison of multiple classifiers.

To meet the project requirements, we implemented a full classification pipeline that: performs exploratory data analysis (EDA), splits the dataset into stratified training, validation, and test sets using `random_state = TeamID` so results can be replicated, trains four required models (a baseline classifier, Decision Tree, Random Forest, and an additional model of our choice), performs model selection using validation F1-score with at least three values for key hyperparameters, and evaluates the best version of each model on the test set using Accuracy, Precision, Recall, F1-score, and AUC.

In addition, the project requires an outlier detection experiment to examine whether unusually behaving data points negatively affect model performance. For this, we used KMeans clustering on nutritional features to form nutritional profile clusters, applied the Elbow Method to guide selection of the number of clusters, ranked points by their distance to cluster centroids, removed the top 5% most outlier like training samples, and retrained all classifiers using the updated training set. This allowed us to directly compare model performance before vs after outlier removal while keeping the validation and test splits fixed, ensuring a fair evaluation.

Section 1: Team Collaboration

Our team consisted of **three members**, and we collaborated using a mix of shared drives, GitHub, and group meetings. We divided the project responsibilities to make sure each part of the EDA and preprocessing was completed efficiently.

Who Did What

- **Member 1 (Livan Miranda):**
 - Set up the project structure and repository.
 - Performed initial data inspection and cleaning.

- Wrote Python scripts for EDA and generated charts, tables, and summary statistics.
 - Oversaw integration of all sections into the final report.
- **Member 2 (Valentina Kloster:)**
 - Conducted attribute-level analysis for variables 1–10.
 - Checked for missing values, outliers, and recommended preprocessing strategies.
 - Assisted with rewriting explanations and formatting findings.
- **Member 3 (Mora Dominguez:)**
 - Worked on attribute-level analysis for variables 11–19.
 - Validated distributions, correlations, and created visualizations.
 - Helped compile and organize the final summary section.

How We Collaborated

- We met several times during the week to review everyone's progress and align preprocessing methods so all attributes were handled consistently.
- We shared files through **Google Drive** and used a **Github repository** so all members could test code and verify results.
- When one member finished early, they assisted others with plots, formatting, or checking calculations.

Issues We Faced

- At the beginning, we had different interpretations of how detailed each attribute analysis needed to be, so we had to re-align and rewrite some sections for consistency.

- Some preprocessing decisions (e.g., handling outliers or categorical consolidation) required discussion to ensure we applied the same logic across all attributes.
- Scheduling meetings was sometimes challenging due to conflicting class/work schedules, but we resolved this through online collaboration.

Experimental Setup (SECTION 2).....	4
Data Split.....	7
Preprocessing.....	7
Classification Models(SECTION 3).....	8
Decision Tree.....	8
Random Forrest.....	9
XGBOOST.....	9
Evaluation(SECTION 4).....	11
Model Results (Scenario 1).....	11
Outlier Detection (KMeans on Nutritional Profiles).....	12
Model Results (Scenario 2: Training Outliers Removed).....	14
Overall Discussion and Final Conclusion(SECTION 5).....	17
Tools and Sources.....	17
Summary.....	17

Experimental Setup (Data Split, Preprocessing, and Tuning)

Target ConstructionThe dataset originally includes the NOVA ultra-processing classification (f_FPro_class) with four categories (0–3).

We transformed it into a **binary classification target**:

- **1 = non-UPF** (classes 0, 1, 2)
- **0 = UPF** (class 3)

This matches the project requirement for binary ultra-processed food detection.

2. Feature Selection & Normalization

We selected two groups of predictors:

1. **Numeric nutritional features** (e.g., fat, sodium, sugars, fiber).
 - Missing values → imputed with **median**
 - Values → **StandardScaler** normalization
2. **Categorical features** (name, store, food category, brand)
 - Missing values → most-frequent imputation
 - Encoded using **One-Hot Encoding**, with `handle_unknown="ignore"`
 - These produce multiple binary columns but are treated as **a single attribute per original feature** in the table.

ID columns such as **original_ID** were removed before training.

3. Handling Class Imbalance

Your code includes both **RandomOverSampler** and **SMOTE**, but the final configuration uses:

- **SMOTE oversampling on the training split only**
- No oversampling on validation or test sets
- Baseline model uses **no SMOTE**

This ensures:

- Balanced training data
- Unbiased validation/test evaluation
- Synthetic minority examples only influence model fitting, not scoring

Data & Experimental Preparation

The dataset contains product identifiers, retailer/category/brand metadata, and nutritional measurements. The modeling goal is a binary classification problem that predicts whether a product is non-ultra-processed (positive class) versus ultra-processed (negative class), using the provided FPro processing label as ground truth after mapping. The original label distribution is highly imbalanced, and after mapping to binary form the dataset contains 12,870 ultra-processed (0) and 3,754 non-ultra-processed (1) items (about 23% positive class).

NAME	MEANING	IS NORMALIZED	TYPE
store (one-hot)	Retailer source (e.g., Walmart/Target/WholeFoods)	No	Categorical (one-hot)
food_category (one-hot)	Product category label	No	Categorical (one-hot)
brand_reduced (one-hot)	Brand, reduced by grouping rare brands into “Other” and missing into “Unknown”	No	Categorical (one-hot)
price	Item price	Yes (standardized)	Numeric (continuous)
price_percal	Price per calorie	Yes (standardized)	Numeric (continuous)
package_weight	Package weight	Yes (standardized)	Numeric (continuous)
Protein	Protein amount	Yes (standardized)	Numeric (continuous)
Total Fat	Total fat amount	Yes (standardized)	Numeric (continuous)
Carbohydrate	Total carbohydrates	Yes (standardized)	Numeric (continuous)
Sugars, total	Total sugar	Yes (standardized)	Numeric (continuous)

Fiber, total dietary	Dietary fiber	Yes (standardized)	Numeric (continuous)
Calcium	Calcium amount	Yes (standardized)	Numeric (continuous)
Iron	Iron amount	Yes (standardized)	Numeric (continuous)
Sodium	Sodium amount	Yes (standardized)	Numeric (continuous)
Cholesterol	Cholesterol amount	Yes (standardized)	Numeric (continuous)
Fatty acids, total saturated	Saturated fat	Yes (standardized)	Numeric (continuous)
f_FPro_class	Original NOVA class (0–3)	No	Target

Data Split

The data was split into Train/Validation/Test with stratification on the binary target and random seed = 10. The resulting split sizes were Train = 9,974 rows, Validation = 3,325 rows, and Test = 3,325 rows. The positive-class rate was approximately 22.6% across all three splits (stratification preserved the class ratio). Validation = 3,325 rows, and Test = 3,325 rows. The positive class rate was approximately 22.6% across all three splits (stratification preserved the class ratio)

Preprocessing

The preprocessing pipeline was fit on the training set only and then applied to validation and test. Numeric features were standardized, and categorical features were one-hot encoded. After preprocessing, the feature matrix contained 11,984 features for the original scenario (and 11,409 features after outlier removal, because the training set used to fit the encoder changed).

Overall

Data preprocessing and experimental setup were designed to be fully reproducible and consistent across models. We removed identifier text fields (e.g., product name) from modeling to avoid high-cardinality sparsity, and we applied one-hot encoding to the categorical fields (store,

food_category, and reduced-brand) while applying standardization to all numeric nutritional fields using statistics fit on the training set only.

We split the dataset into stratified train/validation/test partitions with a fixed random seed equal to the TeamID, resulting in Train = 9,974, Validation = 3,325, and Test = 3,325 examples (with the class ratio preserved across splits (approximately 23% positive class).) Because the positive class is the minority and the assignment emphasizes F1 score, we addressed class imbalance by applying SMOTE only to the training split for the learning-based models (Decision Tree, Random Forest, XGBoost). We did not apply SMOTE to the baseline classifier. Model selection was performed by choosing the hyperparameters that maximized validation F1, and final model comparison was performed on the held-out test split.

Classification Models

A simple baseline, non-learning classifier is used as a point of reference by always predicting the most frequent class in the training set. It works by computing the majority class label during fitting and then outputting that same label for every input instance at prediction time, regardless of the feature values.

In this project, we used the majority baseline to establish a minimum performance level and to illustrate what happens when no meaningful patterns are learned from the dataset features. Although the baseline achieves an accuracy of approximately 0.774, its precision, recall, and F1-score are 0 because it never predicts the positive class. This highlights why F1-score is a more appropriate metric than accuracy for imbalanced classification problems, where predicting only the majority class can appear right by accuracy while failing to detect the minority class entirely.

Decision Tree

A Decision Tree is a supervised learning model that predicts a class label by learning a set of interpretable if-then rules that split the feature space into regions associated with each class. It works by recursively selecting splits that reduce node impurity and stopping based on constraints such as maximum depth and the minimum number of samples required to split a node, which help control overfitting.

In this project, we tuned max_depth and min_samples_split and trained the model on the SMOTE-balanced training data to reduce bias toward the majority class and improve minority-class detection. The Decision Tree substantially improves F1-score compared to the

baseline, but it performs worse than the ensemble methods, which suggests that a single tree may not capture the full complexity of the feature relationships after one-hot encoding and may underfit patterns that benefit from averaging or boosting.

Random Forest

A Random Forest is an ensemble learning method that improves predictive performance by averaging the outputs of many decision trees trained on different versions of the data. It works using bagging: each tree is fit on a bootstrapped sample of the training set, and at each split the algorithm considers only a random subset of features, which reduces correlation between trees and lowers variance.

In this project, we tuned `n_estimators` and `max_depth` and trained the forest on SMOTE-balanced training data to strengthen learning for the minority class. The Random Forest achieves very high training performance (often approaching a perfect fit), while validation and test F1-scores are lower, which indicates mild overfitting; however, it still produces strong test performance overall and clearly outperforms the baseline and the single Decision Tree in most settings.

XGBoost (Gradient Boosted Trees)

XGBoost (Gradient Boosted Trees) is a gradient boosting algorithm that builds an ensemble of trees sequentially, where each new tree is trained to correct the errors made by the current model, ultimately producing a strong boosted predictor. It works by optimizing a differentiable loss function (logloss for binary classification) using additive decision trees, with regularization and learning-rate shrinkage to control complexity and improve generalization.

In this project, we tuned `n_estimators`, `max_depth`, and `learning_rate`, used CPU-friendly training settings (`tree_method='hist'`), and selected the best configuration using validation F1-score. XGBoost achieved the best overall performance across both scenarios, producing the highest validation F1 and the strongest test F1/AUC, making it the selected best model for this dataset.

Overall Models (Models Summary)

The baseline classifier is a simple non-learning reference method that always predicts the most frequent class observed in the training data. It does not fit parameters from features; instead, it calculates the majority class label during fit and outputs that same label for every instance at prediction time. In this project, we used the baseline to establish what performance looks like when the model learns nothing from the nutritional and categorical features. This baseline achieves an accuracy near the majority class proportion (about 0.774), but it produces precision/recall/F1 of 0 due to the fact that it never predicts the positive class, which demonstrates why F1 is more informative than accuracy for imbalanced classification.

The Decision Tree model is a supervised learning method that predicts the class label using a hierarchy of if-then splitting rules. It recursively selects feature splits that reduce impurity and stops growth based on constraints such as maximum depth and minimum samples per split. In this project, we trained the tree on the SMOTE-balanced training data and tuned $\text{max_depth} \in \{3, 5, 7\}$ and $\text{min_samples_split} \in \{2, 10, 20\}$; with a fixed random seed for reproducibility we select the best configuration by validation F1. Decision trees improved substantially over the baseline because they can learn nonlinear decision boundaries over the one-hot and standardized features; however, as a single model they can still be less stable than ensemble methods on high-dimensional one-hot encoded representations.

The Random Forest model is an ensemble that averages many decision trees trained on bootstrapped samples, while also restricting each split to consider only a subset of features. This strategy reduces variance and typically improves generalization compared to a single tree. In this project, we trained Random Forest models on the SMOTE balanced training data and tuned $\text{n_estimators} \in \{100, 200, 300\}$ and $\text{max_depth} \in \{5, 10, \text{None}\}$, again selecting the best setting by validation F1 under a fixed random seed. We observed that the forest can fit the training set extremely well (very good training F1), while validation/test performance remains slightly lower, which suggests some overfitting but still strong generalization relative to the single-tree baseline.

XGBoost (Gradient Boosted Trees) is a boosting algorithm that builds an additive sequence of decision trees, where each new tree focuses on correcting the errors made by the previous ensemble. It optimizes a differentiable objective and typically achieves strong performance by combining shrinkage with model complexity controls and additional regularization. In this project, we tuned $\text{n_estimators} \in \{200, 300, 400\}$, $\text{max_depth} \in \{3, 5, 7\}$, and $\text{learning_rate} \in \{0.05, 0.1, 0.2\}$, used CPU-friendly settings (e.g., histogram-based tree building), and selected the best model by validation F1. Across both the original and outlier-removed scenarios, XGBoost achieved the strongest overall validation and test performance, making it the final recommended model for this dataset.

EVALUATION

Model Results (Scenario 1: Original Training Data)

The table below shows, for the original dataset, the best hyperparameter combination per model and the corresponding train/validation/test F1 and test accuracy / ROC-AUC.

scenario	model	split	params	accuracy	precision	recall	f1	roc_auc
original	baseline_majority	test	{}	0.7741353383458647	0.0	0.0	0.0	
original	decision_tree	test	{'max_depth': 7, 'min_samples_split': 2}	0.916390977443609	0.7678369195922989	0.9027962716378163	0.8298653610771114	0.952239541786812
original	random_forest	test	{'n_estimators': 300, 'max_depth': None}	0.9491729323308271	0.9193083573487032	0.8495339547270306	0.8830449826989619	0.9866795063199857
original	xgboost	test	{'n_estimators': 200, 'max_depth': 7, 'learning_rate': 0.1}	0.9627067669172933	0.9066147859922179	0.9307589880159787	0.9185282522996058	0.9922289576084515

Table 1. Test-set performance for each classifier (Original training data).

Table 1 summarizes the four classifiers on the same held out test set using the best hyperparameters selected by validation F1. The baseline classifier has F1=0 as expected. The Decision Tree performs reasonably (test F1=0.830), but ensemble methods perform better. Random Forest improves test F1 to =0.883 with strong AUC=0.987. XGBoost achieves the best test performance with F1=0.919 and AUC=0.992.

Conclusion (Scenario 1): The best model on the original training data is XGBoost, based on validation F1 and confirmed by the highest test F1/AUC.

Train/Validation/Test F1 and AUC (Scenario 1)

Model	Train F1	Val F1	Test F1	Train AUC	Val AUC	Test AUC
baseline_majority	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
decision_tree	0.842942	0.815085	0.829865	0.969293	0.947562	0.952240
random_forest	1.000000	0.883436	0.883045	1.000000	0.986475	0.986680
xgboost	0.997119	0.932281	0.918528	0.999980	0.993034	0.992229

Table 2. Train/Validation/Test F1 and AUC (Scenario 1)

Table 2 reports F1 and AUC on all three splits for each classifier. The Random Forest and XGBoost show very high training scores, which indicates strong learning capacity and potential overfitting. However, validation and test performance remain high, and XGBoost maintains the best generalization metrics.

Outlier Detection (KMeans on Nutritional Profiles)

KMeans clustering is an unsupervised learning method that groups foods with similar nutritional profiles into K clusters, and in this project we used it for outlier detection by measuring how far each training point is from its assigned cluster centroid.

We implemented it by selecting numeric nutritional features, dropping rows with missing numeric values for the clustering step, and standardizing the remaining values using StandardScaler so features are on comparable scales. We then computed the Within Cluster Sum of Squares (inertia) for $k = 1 \dots 10$ to generate an elbow plot, and for automation and consistency across runs we selected $k = 4$, fit the KMeans model, computed each point's minimum distance to a centroid, ranked points by distance, and removed the top 5% farthest points as outliers.

The motivation is that outliers may represent unusual products (extreme nutritional values or rare combinations) that could introduce noise into learning; therefore, after removing the top 5% most

outlier-like training examples (498 out of 9,974 records), we retrained and evaluated the same four classifiers to examine how outlier removal affects test performance.

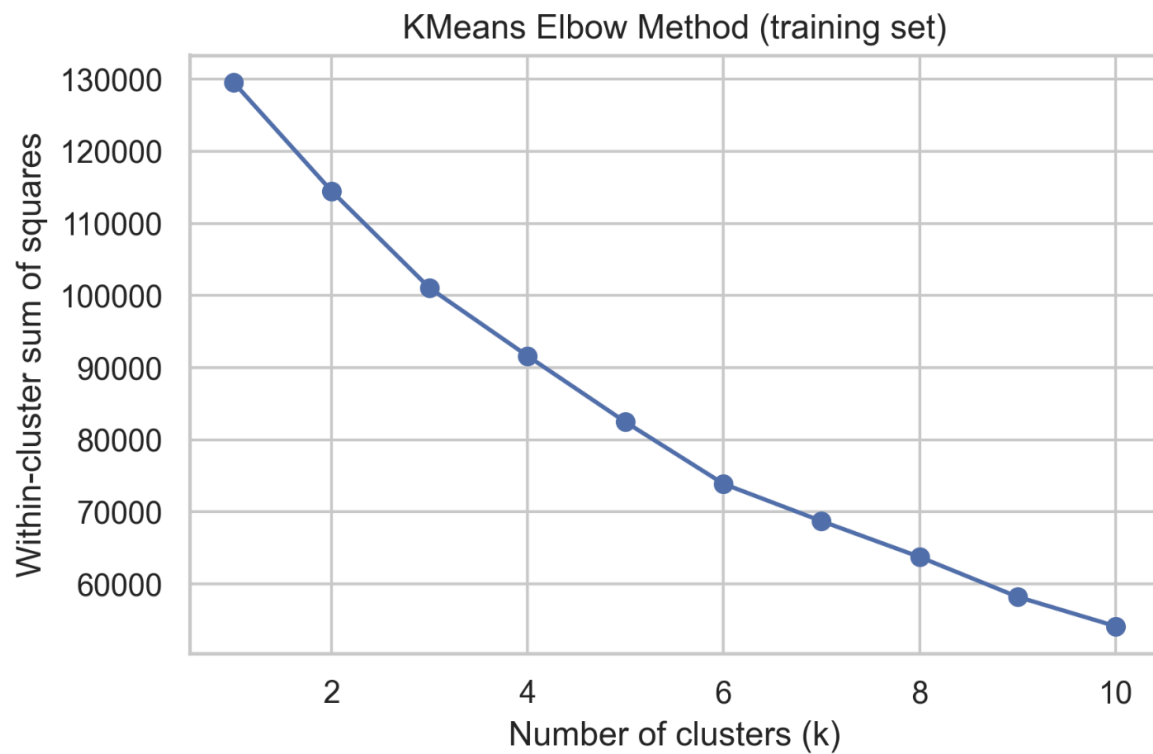


Figure 1. KMeans Elbow Method (training set).

Figure 1 shows the elbow curve of Within Cluster Sum of Squares as the number of clusters increases. The curve decreases rapidly for small k and then flattens, suggesting diminishing returns after a small number of clusters. We selected $k=4$ as a reasonable balance between compact clusters and model simplicity.

Model Results (Scenario 2: Training Outliers Removed)

scenario	model	split	params	accuracy	precision	recall	f1	roc_auc
outlier_removed	baseline_majority	test	{}	0.7741353383458647	0.0	0.0	0.0	
outlier_removed	decision_tree	test	{'max_depth': 7, 'min_samples_split': 20}	0.8965413533834586	0.7087179487179487	0.9201065246338216	0.8006952491309386	0.9435282870702312
outlier_removed	random_forest	test	{'n_estimators': 200, 'max_depth': None}	0.9452631578947368	0.9177679882525698	0.8322237017310253	0.8729050279329609	0.9856391943608988
outlier_removed	xgboost	test	{'n_estimators': 400, 'max_depth': 7, 'learning_rate': 0.05}	0.9621052631578947	0.9084967320261438	0.9254327563249002	0.91688654353562	0.991440575994504

Table 3. Test-set performance for each classifier (Outliers removed from training set).

Table 3 summarizes the retrained classifiers after removing 5% outlier-like points from the training set. Overall, test performance does not improve. For example, XGBoost decreases slightly in test F1 compared to Scenario 1 ($=0.9185 \rightarrow =0.9169$), and Random Forest also decreases slightly ($=0.8830 \rightarrow =0.8729$).

Train/Validation/Test F1 and AUC (Scenario 2)

Model	Train F1	Val F1	Test F1	Train AUC	Val AUC	Test AUC
baseline_majority	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
decision_tree	0.815795	0.787529	0.800695	0.973058	0.943011	0.943528
random_forest	1.000000	0.869924	0.872905	1.000000	0.984785	0.985639
xgboost	0.998278	0.923784	0.916887	0.999995	0.991966	0.991441

Table 4. F1 and AUC across Train/Validation/Test for each classifier (Outlier-removed scenario).

Conclusion (Scenario 2)

In Scenario 2, removing outlier-like points from the training set did not improve generalization on the test set for this dataset. Although the outlier removal process eliminated 498 training

records (top 5% by distance to their KMeans centroid) and slightly changed the feature space learned by the models, the best-performing classifier (XGBoost) achieved nearly the same test performance as the original scenario. This indicates that the outlier of nutritional profiles were not significantly harming the learning process, and that the models (especially XGBoost) were already robust enough to handle those atypical observations without a measurable gain from removing them.

Evaluation Explanation

To evaluate each classifier, we reported Accuracy, Precision, Recall, F1-score, and AUC. Accuracy measures the overall fraction of correct predictions, but it can be misleading when classes are imbalanced because a model can achieve high accuracy by mostly predicting the majority class. Precision measures how many predicted positives are truly positive, which matters when false alarms are costly. Recall measures how many true positives are successfully identified, which matters when missing positives is costly. The F1-score is the mean of precision and recall, and it provides a single balanced measure that is more informative than accuracy under class imbalance. Finally, AUC evaluates how well a model ranks positive examples above negative examples across all thresholds, giving a threshold-independent view of separability between classes.

Model Selection and Hyperparameter Testing Procedure

For each model, we performed model selection using the validation set. Specifically, we trained each model on the training split (after applying preprocessing and handling class imbalance using SMOTE for models), evaluated candidate hyperparameter combinations on the validation split, and selected the best configuration based on the highest validation F1-score. After selecting the best configuration for each model, we reported performance on the training, validation, and test splits. The train/validation/test splits were stratified using the TeamID as the random seed to ensure reproducibility and consistent comparisons across models.

Hyperparameter Grids Tested

For each train model, we tested at least three values for each key hyperparameter. For the Decision Tree, we tuned `max_depth` and `min_samples_split`. For the Random Forest, we tuned `n_estimators` and `max_depth`. For XGBoost, we tuned `n_estimators` and `max_depth` as well as `learning_rate`. The baseline classifier did not use hyperparameter tuning because it does not learn from features; it always predicts the majority class in the training set.

Hyperparameter Testing Results and Best-Configuration Selection

For each model, we compared hyperparameter combinations using validation F1-score and selected the configuration with the highest validation F1 as the best version of that model. This decision rule was applied consistently across both the original training set and no outlier training set. The selected best configuration per model is explicitly shown in the Results tables (params), and these best configurations are the ones used for the final test model comparison. This approach ensures that test results reflect a fair comparison of models after each model has been tuned using the validation set only.

Final Model Comparison on the Test Set

After selecting the best hyperparameter configuration for each classifier using validation F1-score, we compared models on the test set using Accuracy, Precision, Recall, F1, and AUC. Across both scenarios, XGBoost achieved the strongest overall performance, producing the highest test F1 and AUC. Random Forest also performed strongly but showed evidence of mild overfitting because training metrics were perfect while validation and test were lower. The Decision Tree improved substantially over the baseline but underperformed the ensemble methods, suggesting that a single tree was not expressive enough to match the boosted or bagged ensembles in this high-dimensional one-hot encoded feature space. The baseline majority classifier achieved moderate accuracy but had precision/recall/F1 of 0 because it never predicted the positive class; this demonstrates why accuracy is not an appropriate primary metric for imbalanced classification.

Effect of Outlier Removal

To examine whether removing outlier points improves generalization, we compared the original training set to the altered training set (that had top 5% farthest points removed based on KMeans centroid distance.) Overall, removing outliers did not improve test generalization. XGBoost achieved pretty much the same test performance in both scenarios, while Random Forest and Decision Tree slightly decreased in test F1 after outlier removal. This indicates that the detected outliers of nutritional profiles were not significantly harming model learning, and the strongest models (especially XGBoost) were already robust enough to handle atypical observations without a measurable gain from removing them.

Overall Discussion and Final Conclusion

Across both scenarios, XGBoost was the best performing model by validation F1 and achieved the highest test F1 and AUC. Outlier removal reduced the size of the training set and slightly

changed the learned preprocessing representation, but did not improve test performance; therefore, the original training data is preferred for this task.

Final selected model: XGBoost trained on the original training data with best parameters.

Tools and Sources

We implemented the pipeline in Python using: scikit-learn (preprocessing, trees, metrics, KMeans), imbalanced-learn (SMOTE), xgboost (XGBClassifier), pandas/numpy (data handling), and matplotlib (plots). We also used an LLM (ChatGPT) to help debug code errors and improve report writing.

Conclusion

This project implemented and evaluated four classification models to predict the binary target using a standardized, reproducible pipeline. We created a simple baseline majority classifier as a reference point, then trained three learning based models: a Decision Tree, a Random Forest, and an XGBoost classifier. The dataset was split into stratified training/validation/test sets using `random_state = TeamID (10)` to ensure consistent class proportions across splits and reproducible experiments. Hyperparameter tuning was performed using the validation set F1-score, and final performance was reported on the test set using Accuracy, Precision, Recall, F1, and AUC.

Across the original data scenario, the XGBoost model produced the strongest results overall, achieving test F1 ≈ 0.9185 and test AUC ≈ 0.9922 , clearly outperforming the Decision Tree and Random Forest and massively improving over the baseline (which has F1 = 0 because it predicts only the majority class). For the outlier experiment, we applied KMeans clustering on numeric nutritional features and used the Elbow Method (Figure 1) to guide the choice of a reasonable number of clusters (we used $k = 4$ for automation). We then ranked points by distance to centroid and removed the top 5% most “outlier-like” training samples (498 points). After retraining the same four models using the updated training set, XGBoost remained the best model, with test F1 = 0.9169 and test AUC = 0.9914.

Overall, removing outliers produced very small changes in performance: the outlier removed scenario did not significantly improve generalization compared to the original scenario, and the best overall performance remained essentially the same.

The key conclusion is that :

(1) learning-based models (especially XGBoost) substantially outperform the baseline

(2) the model performance is already very strong on this dataset (high F1 and AUC)

(3) outlier removal (top 5%) did not meaningfully improve results, meaning that the nutritional profiles were not strongly harming the classifier's ability to generalize on the test set.